

山东大学



网络空间安全创新创业实践实验报告

作业 3: AES-CTR 模式优化实现与性能对比

姓 名: 辛汶原 学 号: 202300460115 班 级: 网安 1 班
姓 名: 王志歌 学 号: 202300460119 班 级: 网安 2 班
姓 名: 刘鹤扬 学 号: 202300180102 班 级: 网安 1 班
学 院: 网络空间安全学院

2026 年 7 月 24 日

目录

目录 Table of Contents	I
摘要	1
1 组员分工	1
2 实验目的	2
3 实验环境配置	3
3.1 硬件	3
3.2 软件	3
3.3 项目目录	3
4 实验原理	4
4.1 AES-CTR 模式简介	4
4.2 T-table 软件优化原理	4
4.3 AES-NI 与 SIMD 硬件优化原理	4
5 实验实现	5
5.1 总体流程	5
5.2 核心代码实现	5
5.2.1 基础结构与辅助函数	5
5.2.2 T-table 软件优化实现	5
5.2.3 AES-NI 与 4 路循环展开硬件优化实现	7
5.2.4 主控制流与正确性验证	8
6 实验结果与分析	10
6.1 终端执行输出示例	10
6.2 一致性与正确性分析	10
7 结论	11

摘要

高级加密标准 (AES) 在实际应用中广泛采用 CTR (计数器) 模式, 以支持并行加密和流加密特性。为了在不同环境中实现高效加密, 通常需要针对特定的执行环境进行实现优化。本实验研究了 AES-CTR 模式的两种主流优化策略: 基于 T-table 的软件优化, 以及基于 AES-NI 硬件指令集的优化。

本实验中, 我们首先使用 C++ 构建了基于 T-table (Lookup Tables) 的软件实现。该实现通过将 SubBytes、ShiftRows 以及 MixColumns 融合为 32 位查找表操作, 大幅降低了每轮加密的运算开销。进一步, 我们结合现代 CPU 的 SIMD 硬件指令集 (AES-NI 及 SSSE3), 实现了基于 4 路循环展开 (4-Way Loop Unrolling) 的 CTR 模式优化版本。该版本通过 `pshufb` 解决大小端转换问题, 并同时利用 4 路并行掩盖 AES 指令的延迟。

实验在同一套密钥和初始向量 (IV) 环境下, 验证了软件优化结果与硬件优化结果的严格一致性, 并通过加解密交叉验证, 保证了实现的数学正确性与工程正确性。本报告详细阐述了上述两套优化的基本原理、核心代码实现以及验证流程。

关键词: AES-CTR; T-table 优化; AES-NI 硬件指令集; SIMD; 循环展开

1 组员分工

本次实验的小组分工如表 1 所示。

姓名	学号	分工
刘鹤扬	202300180102	未参与当前实验, 负责任务 4 的 X86 部分
辛汶原	202300460115	未参与当前实验, 负责任务 4 的 ARM 部分
王志歌	202300460119	独立完成实验 3, 代码及报告撰写

表 1: 小组分工

2 实验目的

本实验旨在通过实际编码实现，理解 AES-CTR 模式的工作机制及其在软件和硬件层面的性能优化策略，具体目的包括：

1. **理解软件优化原理。**掌握如何将 AES 轮函数中的非线性替换 (SubBytes)、行移位 (ShiftRows) 和列混淆 (MixColumns) 步骤，整合为查找表 (T-table) 访问，以减小计算量。
2. **掌握硬件加速指令集。**学习并应用 Intel AES-NI 硬件指令集 (如 `_mm_aesenc_si128` 等) 与 SSSE3 字节洗牌指令 (`_mm_shuffle_epi8`)，将底层的密码操作交由专用硬件执行。
3. **探索并行化技术。**在 CTR 模式不具有数据依赖性的前提下，实现 4 路循环展开 (Loop Unrolling)，以隐藏指令延迟，发挥指令集的最大流水线吞吐能力。
4. **正确性验证。**设计严谨的对比实验，验证不同优化方式产生的结果一致，并能够成功解密密文，确保算法逻辑无误。

3 实验环境配置

3.1 硬件

- CPU: 支持 SSSE3 及 AES-NI 指令集的现代 CPU (如 Intel Core/AMD Ryzen 系列)
- 内存: ≥ 16 GB
- 操作系统: Windows 11

3.2 软件

- 编程语言: C++ (C++11 及以上标准)
- 编译器: 支持 `<immintrin.h>` 和 SIMD 优化指令的编译器 (如 GCC, Clang, MSVC)

3.3 项目目录

```
D:\Desktop\...\ 作业2_ECDSA签名伪造分析
|-- 3.cpp          <-- 实验核心代码
|-- README.md
\-- report/        <-- 实验报告 LaTeX 源码
    |-- report.tex
    |-- SDUstyle.sty
    \-- imgs/
```

4 实验原理

4.1 AES-CTR 模式简介

计数器模式 (Counter Mode, CTR) 将块密码转变为流密码。在 CTR 模式下, 对于每个明文块, 不直接将明文输入块密码算法, 而是对一个随数据块递增的“计数器” (Counter) 结合初始向量 (IV) 进行加密, 得到伪随机密钥流 (Keystream)。然后将该密钥流与明文块进行异或 (XOR), 从而得到密文。由于计数器的生成和加密不依赖于前一个密文块, 因此 CTR 模式支持高度并行化。

4.2 T-table 软件优化原理

传统的 AES 轮函数包含 SubBytes、ShiftRows、MixColumns 和 AddRoundKey 四个步骤。在 32 位处理器上, 逐字节处理效率较低。T-table 优化策略通过预先计算将 SubBytes、ShiftRows 和 MixColumns 合并为 4 个大小为 256 (共占 $4 \times 1\text{KB}$ 内存) 的 32 位查找表 (T_0, T_1, T_2, T_3)。在执行 1 到 9 轮的加密时, 每个列的计算可以通过简单的 4 次查表操作与 4 次异或操作完成:

$$s'_j = T_0[s_{0,j}] \oplus T_1[s_{1,j}] \oplus T_2[s_{2,j}] \oplus T_3[s_{3,j}] \oplus k_j$$

从而利用存储换取计算, 大幅度提升了纯软件环境下的执行速度。

4.3 AES-NI 与 SIMD 硬件优化原理

AES-NI (Advanced Encryption Standard New Instructions) 是 x86 架构下的 AES 加速指令扩展, 将 AES 轮计算实现在芯片级流水线中。

- `_mm_aesenc_si128` 执行一次常规的 AES 轮操作。
- `_mm_aesenclast_si128` 执行无需列混淆的最后一轮。

在 CTR 模式下, 计数器是以大端序递增的, 而 x86 处理器为小端序。因此我们需要借助 SSE3 提供的字节重排指令 `_mm_shuffle_epi8` (即 `pshufb`), 在大端与小端格式间高效切换, 避免传统 `bswap` 标量指令导致的性能瓶颈。

由于单挑 `aesenc` 指令存在数个周期的延迟, 我们利用 CTR 的无状态特性, 同时加载并递增 4 个计数器模块 (4-Way Loop Unrolling)。四路并行能够充分填满指令流水线, 彻底隐藏执行延迟。

5 实验实现

5.1 总体流程

实验的实现按如下方式组织：

1. 编写 `AESTableOpt` 类：在构造函数中预生成 T_0, T_1, T_2, T_3 表，提供单块加密和 CTR 模式实现。
2. 编写 `AES_NI_CTR_Optimized` 类：利用指令集进行密钥扩展，使用 `_mm_shuffle_epi8` 进行端序转换，并实现每次处理 64 字节的 4 路循环展开函数。
3. `main` 函数：生成测试数据，分别调用两类方法，然后通过逐字节比对，验证软硬件优化结果的严格一致性，最后对硬件密文进行再次解密验证。

5.2 核心代码实现

本实验的核心代码位于 `3.cpp`，我们将其按逻辑划分为四个主要模块进行详细解析：基础结构与辅助函数、T-table 软件优化实现、AES-NI 硬件指令优化实现、以及主控制流与正确性验证。

5.2.1 基础结构与辅助函数

首先引入必要的头文件，其中 `<immintrin.h>` 提供了调用硬件 SIMD 指令（如 SSE, SSSE3, AES-NI）的内联函数支持。辅助函数 `printHex` 用于在终端打印十六进制格式的数据块，方便我们在控制台核对加密过程中的内部状态。

```
1 #include <iostream>
2 #include <vector>
3 #include <cstdint>
4 #include <cstring>
5 #include <iomanip>
6 #include <string>
7 #include <immintrin.h> // 现代 CPU 指令集支持: SSE, SSSE3, AES-NI 等
8
9 using namespace std;
10
11 // 打印十六进制格式的数据块，方便中间状态的核对
12 void printHex(const string& label, const uint8_t* data, size_t length) {
13     cout << label << " (" << length << " bytes):" << endl;
14     for (size_t i = 0; i < length; i++) {
15         cout << hex << setw(2) << setfill('0') << (int)data[i] << " ";
16         if ((i + 1) % 16 == 0) cout << endl;
17     }
18     cout << dec << endl;
19 }
```

Listing 1: 基础头文件与辅助打印函数

5.2.2 T-table 软件优化实现

`AESTableOpt` 类实现了无硬件加速环境下的高级软件优化。其核心在于将 AES 轮函数中的非线性替换（SubBytes）、行移位（ShiftRows）和列混淆（MixColumns）步骤合并查表。

1. T-table 的预计算 在类的构造函数中, 利用原始的 SBOX 以及伽罗瓦域 (Galois Field) 乘法函数 (mul2 和 mul3), 提前计算出 4 个容量为 256 的 32 位查找表 T_0, T_1, T_2, T_3 。

```
1 class AESTableOpt {
2 private:
3     uint32_t T0[256], T1[256], T2[256], T3[256];
4     uint32_t rk[44]; // AES-128 扩展密钥: 44 个 32 位字
5     const uint8_t SBOX[256] = { /* ... 256 字节原始 S 盒 ... */ };
6
7     uint8_t mul2(uint8_t a) { return (a & 0x80) ? ((a << 1) ^ 0x1B) : (a << 1); }
8     uint8_t mul3(uint8_t a) { return mul2(a) ^ a; }
9
10 public:
11     AESTableOpt() {
12         for (int i = 0; i < 256; i++) {
13             uint8_t s = SBOX[i], s2 = mul2(s), s3 = mul3(s);
14             T0[i] = (s2 << 24) | (s << 16) | (s << 8) | s3;
15             T1[i] = (s3 << 24) | (s2 << 16) | (s << 8) | s;
16             T2[i] = (s << 24) | (s3 << 16) | (s2 << 8) | s;
17             T3[i] = (s << 24) | (s << 16) | (s3 << 8) | s2;
18         }
19     }
```

Listing 2: AESTableOpt 类的 T-table 初始化

2. 密钥扩展与块加密 密钥扩展函数 SetKey 按照标准的 AES 流程利用 RCON 和 SBOX 进行密钥轮扩展。在单块加密 encryptBlock 中, 输入字节首先被重组为 4 个 32 位整型状态 ($s_0 \sim s_3$)。前 9 轮循环只需要位移和读取四个表; 第 10 轮因为没有列混淆, 回退到查询基础 SBOX 并异或最后一步轮密钥。

```
1 void encryptBlock(const uint8_t* in, uint8_t* out) {
2     // 读取并转换输入为 32 位内部状态
3     uint32_t s0 = (in[0] << 24) | (in[1] << 16) | (in[2] << 8) | in[3];
4     // ... s1, s2, s3 类似处理
5     s0 ^= rk[0]; s1 ^= rk[1]; s2 ^= rk[2]; s3 ^= rk[3];
6
7     // 前 1-9 轮直接查 T-table 聚合操作
8     for (int r = 1; r < 10; r++) {
9         uint32_t t0 = T0[(s0 >> 24) & 0xFF] ^ T1[(s1 >> 16) & 0xFF] ^ T2[(s2 >> 8) & 0xFF] ^ T3[s3 & 0xFF]
        ^ rk[r * 4];
10        // ... 计算 t1, t2, t3 同理
11        s0 = t0; s1 = t1; s2 = t2; s3 = t3;
12    }
13
14    // 第 10 轮处理 (无 MixColumns), 查 SBOX 并按位重组
15    uint32_t out0 = (SBOX[(s0 >> 24) & 0xFF] << 24) | (SBOX[(s1 >> 16) & 0xFF] << 16) | (SBOX[(s2 >> 8) &
        0xFF] << 8) | SBOX[s3 & 0xFF];
16    // ... out1, out2, out3 类似处理
17    out0 ^= rk[40]; // 异或最后一轮轮密钥
18    // ... 最后写回 out 数组
19 }
```

Listing 3: 单块数据的 T-table 轮运算

3. 软件端 CTR 模式封装 对于流数据, 在 EncryptCTR 中管理计数器块, 每次加密生成 16 字节密钥流 stream 并与明文块异或。接着在大端模式下手动将计数器 counter 数组加一。


```
1 void EncryptCTR(const uint8_t* in, uint8_t* out, size_t length, const uint8_t* iv) {
2     uint8_t counter[16];
3     memcpy(counter, iv, 16);
4     uint8_t stream[16];
5     size_t i = 0;
6
7     while (i < length) {
8         encryptBlock(counter, stream); // 加密计数器生成密钥流
9         size_t block_len = (length - i < 16) ? (length - i) : 16;
10        for (size_t j = 0; j < block_len; j++) {
11            out[i + j] = in[i + j] ^ stream[j]; // 与明文异或
12        }
13        i += block_len;
14        // 递增 Counter (大端序)
15        for (int k = 15; k >= 0; k--) {
16            if (++counter[k]) break;
17        }
18    }
19 }
20 };
```

Listing 4: 软件版 CTR 模式封装

5.2.3 AES-NI 与 4 路循环展开硬件优化实现

AES_NI_CTR_Optimized 类调用专用硬件指令集，把密码学繁重操作交给芯片执行，并且利用 4 路并行（4-Way Loop Unrolling）发挥流水线的最大吞吐性能。

1. 基于 AES-NI 的密钥扩展 key_expand 辅助函数以及 SetKey 函数，直接调用 _mm_aeskeygenassist_si128 等 AES 指令在硬件层面生成轮密钥。

```
1 class AES_NI_CTR_Optimized {
2 private:
3     __m128i round_keys[11];
4     inline __m128i key_expand(__m128i key, __m128i keygened) {
5         key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
6         key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
7         key = _mm_xor_si128(key, _mm_slli_si128(key, 4));
8         keygened = _mm_shuffle_epi32(keygened, _MM_SHUFFLE(3, 3, 3, 3));
9         return _mm_xor_si128(key, keygened);
10    }
11 public:
12     void SetKey(const uint8_t* key) {
13         round_keys[0] = _mm_loadu_si128((const __m128i*)key);
14         round_keys[1] = key_expand(round_keys[0], _mm_aeskeygenassist_si128(round_keys[0], 0x01));
15         // ... 同理依序计算 round_keys[2] 至 round_keys[10]
16    }
```

Listing 5: 基于硬件指令集的密钥扩展

2. SSSE3 端序转换与多路指令并行 在加密流程中，利用 _mm_shuffle_epi8 和固定的 bswap_mask 规避 CPU 的端序问题。我们在一个 while 循环内每次处理 64 字节（4 个分组）。4 个 __m128i 状态量 c0 ~ c3 并发执行 _mm_aesenc_si128 指令，由于 4 路计算相互独立，CPU 的乱序执行和流水线机制可以完美掩盖多周期延迟。

```
1 void EncryptCTR_4x(const uint8_t* in, uint8_t* out, size_t length, const uint8_t* iv) {
2     // 利用 pshufb 指令实现小端(x86)与大端的高效转换掩码
3     __m128i bswap_mask = _mm_setr_epi8(15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0);
4     __m128i ctr = _mm_loadu_si128((const __m128i*)iv);
5     ctr = _mm_shuffle_epi8(ctr, bswap_mask);
6     __m128i one = _mm_set_epi64x(0, 1);
7     size_t i = 0;
8
9     while (i + 64 <= length) {
10        // 生成连续的 4 个计数器并恢复大端
11        __m128i c0 = _mm_shuffle_epi8(ctr, bswap_mask); ctr = _mm_add_epi64(ctr, one);
12        __m128i c1 = _mm_shuffle_epi8(ctr, bswap_mask); ctr = _mm_add_epi64(ctr, one);
13        __m128i c2 = _mm_shuffle_epi8(ctr, bswap_mask); ctr = _mm_add_epi64(ctr, one);
14        __m128i c3 = _mm_shuffle_epi8(ctr, bswap_mask); ctr = _mm_add_epi64(ctr, one);
15
16        // 第 0 轮: 与初始密钥进行异或
17        c0 = _mm_xor_si128(c0, round_keys[0]);
18        c1 = _mm_xor_si128(c1, round_keys[0]);
19        c2 = _mm_xor_si128(c2, round_keys[0]);
20        c3 = _mm_xor_si128(c3, round_keys[0]);
21
22        // 1 至 9 轮: AES-NI 硬件轮计算指令, 4路并发执行
23        for (int r = 1; r < 10; ++r) {
24            c0 = _mm_aesenc_si128(c0, round_keys[r]);
25            c1 = _mm_aesenc_si128(c1, round_keys[r]);
26            c2 = _mm_aesenc_si128(c2, round_keys[r]);
27            c3 = _mm_aesenc_si128(c3, round_keys[r]);
28        }
29
30        // 第 10 轮: 无需 MixColumns 的收尾计算
31        c0 = _mm_aesenclast_si128(c0, round_keys[10]);
32        c1 = _mm_aesenclast_si128(c1, round_keys[10]);
33        c2 = _mm_aesenclast_si128(c2, round_keys[10]);
34        c3 = _mm_aesenclast_si128(c3, round_keys[10]);
35
36        // 提取明文, 产生密文并写入外存
37        c0 = _mm_xor_si128(c0, _mm_loadu_si128((const __m128i*)(in + i)));
38        // ... c1, c2, c3 同理提取 ...
39        _mm_storeu_si128((__m128i*)(out + i), c0);
40        // ... 同样存储 c1, c2, c3 ...
41        i += 64;
42    }
43
44    // 处理尾部不足 64 字节的部分 (代码逻辑类似, 处理单路)
45    while (i < length) {
46        // ...
47    }
48 }
49 };
```

Listing 6: SSSE3 字节重排与四路并行加密核心逻辑

5.2.4 主控制流与正确性验证

main 函数负责驱动上述两种策略执行, 并做完备的结果一致性对比与正确性检验。首先初始化硬编码的 16 字节 key 与 iv, 以及 64 字节渐变递增明文数据。紧接着调用软件方法产生密文并保存至 sw_ciphertext, 调用硬件方法产生密文并保存至 hw_ciphertext。程序遍历判断双路密文

数组内容是否完全相同，以此证明软硬一致。接着将得到的硬件密文再次作为输入送入 CTR 函数，判断解密出来的明文是否等同于原始明文。

```
1 int main() {
2     AESTableOpt aes_table;
3     AES_NI_CTR_Optimized aes_ni;
4
5     uint8_t key[16] = { 0x2b,0x7e,0x15,0x16,0x28,0xae,0xd2,0xa6,0xab,0xf7,0x15,0x88,0x09,0xcf,0x4f,0x3c };
6     uint8_t iv[16] = { 0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7,0xf8,0xf9,0xfa,0xfb,0xfc,0xfd,0xfe,0xff };
7
8     uint8_t plaintext[64];
9     for (int i = 0; i < 64; ++i) plaintext[i] = static_cast<uint8_t>(i);
10    uint8_t hw_ciphertext[64], sw_ciphertext[64], decrypted[64];
11
12    // 初始化密钥并分别使用 T-table 及硬件指令进行 CTR 模式加密
13    aes_table.SetKey(key);
14    aes_table.EncryptCTR(plaintext, sw_ciphertext, 64, iv);
15
16    aes_ni.SetKey(key);
17    aes_ni.EncryptCTR_4x(plaintext, hw_ciphertext, 64, iv);
18
19    // 验证软硬件双路结果是否严格一致
20    bool isConsistent = true;
21    for (int i = 0; i < 64; i++) {
22        if (sw_ciphertext[i] != hw_ciphertext[i]) {
23            isConsistent = false;
24            break;
25        }
26    }
27    cout << "T-table与AES-NI密文" << (isConsistent ? "完全一致!" : "存在偏差!") << endl;
28
29    // 对称解密验证: 利用已加密密文重新输入, 得到明文并核对
30    aes_ni.EncryptCTR_4x(hw_ciphertext, decrypted, 64, iv);
31    bool ok = true;
32    for (int i = 0; i < 64; i++) {
33        if (plaintext[i] != decrypted[i]) {
34            ok = false;
35            break;
36        }
37    }
38    cout << "解密" << (ok ? "成功: 解密结果匹配" : "失败!") << endl;
39
40    return 0;
41 }
```

Listing 7: 主控制流与正确性验证逻辑

6 实验结果与分析

实验运行时，预设了 64 字节测试明文（从 0x00 至 0x3f），分别采用 T-table 和 AES-NI 分支进行加密。输出结果如下。

6.1 终端执行输出示例

AES加密双路优化验证

原始明文 (64 bytes):

```
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f
```

T-table CTR 密文 (64 bytes):

(密文十六进制输出省略)

AES-NI + SSSE3(Shuffle) 4路 CTR 密文 (64 bytes):

(密文十六进制输出，与 T-table 结果相同)

T-table与AES-NI密文完全一致！

解密得到的明文 (64 bytes):

```
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
... (省略) ...
```

解密成功：解密结果与原始明文匹配。

6.2 一致性与正确性分析

上述结果表明，硬件加速优化下的输出密文与软件优化的输出结果完全相同。这有力地证明了我们在使用 `pshufb` 指令重排计数器端序时的正确性；同时，后续将硬件加密的密文再次使用硬件进行加密（CTR 模式对称），成功还原得到了完整的明文，证明了加解密过程的理论正确。多路循环展开不仅实现了硬件层的并行调用，没有引入额外的状态错乱问题。

7 结论

本实验通过实现并对比 T-table 软件优化与 AES-NI 硬件优化方法，圆满完成了 AES-CTR 模式的高级实践，得出如下结论：

1. **T-table 优化通过预计算大幅消减了标量运算量。**用空间换取时间，在不支持硬件加密的老旧或受限设备上，依然是最主流的软件加速方案。
2. **AES-NI 硬件指令极大提升了 AES 吞吐量。**借由 CPU 内置指令执行 128 位密码逻辑操作，有效解放了传统的循环移位算力瓶颈。
3. **4 路循环展开（Loop Unrolling）是发挥硬件流水线的关键。**CTR 模式非常适合多路并行，通过对 4 个区块并发编排寄存器访问，可以遮蔽 `_mm_aesenc_si128` 指令的延迟，使得吞吐量达到极致。
4. **实验验证了二者的一致性。**不论底层的加速手法有多大差异，上层输出在密码学层面严格对等，保证了可移植性和算法正确性。

整体而言，本实验加深了我们对“密码算法底层实现优化”的认识。

7 参考文献

- [1] NIST. *Advanced Encryption Standard (AES)*. FIPS PUB 197, 2001.
- [2] Shay Gueron. *Intel Advanced Encryption Standard (AES) New Instructions Set*. Intel Corporation, 2010.